Labsheet 2

1. Write a C++ program to implement Linear Search on an array. Display the position of the key if it is found; otherwise display an appropriate message.

```cpp
#include <iostream>
#include <string>
#include <cctype>
#include <algorithm>
using namespace std;

int main1() {
    int a[100], n, key;
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "Enter key: ";
    cin >> key;

    for (int i = 0; i < n; i++) {
        if (a[i] == key) {
            cout << "Element found at position " << i + 1 << endl;
            return 0;
        }
    }

    cout << "Element not found." << endl;
    return 0;
}
```

2. Write a C++ program to implement Binary Search on a sorted array. Display the position of the key if it is found.

```cpp
int main2() {
    int a[100], n, key;
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter sorted elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "Enter key: ";
    cin >> key;

    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (a[mid] == key) {
            cout << "Element found at position " << mid + 1 << endl;
            return 0;
        }
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }

    cout << "Element not found." << endl;
    return 0;
}
```

3. Write a C++ program to compare Linear Search and Binary Search for a given array and display their results

```
int main3() {
    int a[100], n, key;
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter sorted elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "Enter key: ";
    cin >> key;

    int linearPosition = -1;
    for (int i = 0; i < n; i++) {
        if (a[i] == key) {
            linearPosition = i;
            break;
        }
    }

    int low = 0, high = n - 1;
    int binaryPosition = -1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (a[mid] == key) {
            binaryPosition = mid;
            break;
        }
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
```

```
if (linearPosition != -1)
    cout << "Linear Search: Found at position " << linearPosition + 1 << endl;
else
    cout << "Linear Search: Not found" << endl;

if (binaryPosition != -1)
    cout << "Binary Search: Found at position " << binaryPosition + 1 << endl;
else
    cout << "Binary Search: Not found" << endl;

return 0;
```

4. Write a C++ program to implement Bubble Sort in ascending order. Display the array after each pass.

```cpp
int main4() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1])
                swap(a[j], a[j + 1]);
        }

        cout << "After pass " << i + 1 << ": ";
        for (int j = 0; j < n; j++)
            cout << a[j] << " ";
        cout << endl;
    }

    return 0;
}
```

5. Write a C++ program to implement Bubble Sort in descending order.

```cpp
int main5() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] < a[j + 1])
                swap(a[j], a[j + 1]);

    cout << "Descending order: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

6. Write a C++ program to implement Insertion Sort in ascending order. Display the array after each insertion/pass.

```cpp
int main6() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;

        cout << "After pass " << i << ": ";
        for (int k = 0; k < n; k++)
            cout << a[k] << " ";
        cout << endl;
    }

    return 0;
}
```

7. Write a C++ program to implement Selection Sort in ascending order. Display the minimum element selected in each pass.

```cpp
int main7() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIndex])
                minIndex = j;

        cout << "Minimum selected = " << a[minIndex] << endl;
        swap(a[i], a[minIndex]);
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

8. Write a C++ program to implement Merge Sort using separate functions for merging and sorting.

```cpp
void mergeArray(int a[], int low, int mid, int high) {
    int temp[100];
    int i = low;
    int j = mid + 1;
    int k = low;

    while (i <= mid && j <= high) {
        if (a[i] < a[j])
            temp[k++] = a[i++];
        else
            temp[k++] = a[j++];
    }

    while (i <= mid)
        temp[k++] = a[i++];

    while (j <= high)
        temp[k++] = a[j++];

    for (i = low; i <= high; i++)
        a[i] = temp[i];
}

void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        mergeArray(a, low, mid, high);
    }
}

int main8() {
    int a[100], n;
```

```cpp
    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    mergeSort(a, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

9. Write a C++ program to implement Quick Sort using the last element as the pivot.

```cpp
int partitionArray(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }

    swap(a[i + 1], a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partitionArray(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}

int main9() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    quickSort(a, 0, n - 1);
```

```cpp
    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

10. Write a C++ program to implement Quick Sort and display the array after each partition.

```cpp
int partitionDisplay(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (a[j] < pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }

    swap(a[i + 1], a[high]);

    cout << "After partition: ";
    for (int k = 0; k < high + 1; k++)
        cout << a[k] << " ";
    cout << endl;

    return i + 1;
}

void quickSortDisplay(int a[], int low, int high) {
    if (low < high) {
        int p = partitionDisplay(a, low, high);
        quickSortDisplay(a, low, p - 1);
        quickSortDisplay(a, p + 1, high);
    }
}

int main10() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;
```

```cpp
    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    quickSortDisplay(a, 0, n - 1);

    return 0;
}
```

11. Write a menu-driven C++ program that allows the user to choose Linear Search or Binary Search for a given array.

```cpp
int main11() {
    int a[100], n, key, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "1. Linear Search\n2. Binary Search\nEnter choice: ";
    cin >> choice;

    cout << "Enter key: ";
    cin >> key;

    if (choice == 1) {
        for (int i = 0; i < n; i++) {
            if (a[i] == key) {
                cout << "Found at position " << i + 1 << endl;
                return 0;
            }
        }
        cout << "Not found" << endl;
    }
    else if (choice == 2) {
        int low = 0, high = n - 1;

        while (low <= high) {
            int mid = (low + high) / 2;

            if (a[mid] == key) {
                cout << "Found at position " << mid + 1 << endl;
                return 0;
            }
            else if (key < a[mid])
                high = mid - 1;
            else
                low = mid + 1;
        }

        cout << "Not found" << endl;
    }
    else {
        cout << "Invalid choice" << endl;
    }

    return 0;
}
```

12. Write a menu-driven C++ program that allows the user to choose Bubble Sort, Insertion Sort, or Selection Sort

```cpp
int main12() {
    int a[100], n, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "1. Bubble Sort\n2. Insertion Sort\n3. Selection Sort\n";
    cout << "Enter choice: ";
    cin >> choice;

    if (choice == 1) {
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - i - 1; j++)
                if (a[j] > a[j + 1])
                    swap(a[j], a[j + 1]);
    }
    else if (choice == 2) {
        for (int i = 1; i < n; i++) {
            int key = a[i];
            int j = i - 1;

            while (j >= 0 && a[j] > key) {
                a[j + 1] = a[j];
                j--;
            }

            a[j + 1] = key;
        }
    }
    else if (choice == 3) {
        for (int i = 0; i < n - 1; i++) {
            int minIndex = i;

            for (int j = i + 1; j < n; j++)
                if (a[j] < a[minIndex])
                    minIndex = j;

            swap(a[i], a[minIndex]);
        }
    }
    else {
        cout << "Invalid choice" << endl;
        return 0;
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

13. Write a C++ program to sort an array using Merge Sort and then search for a given element using Binary Search.

```
int main13() {
    int a[100], n, key;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    mergeSort(a, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;

    cout << "Enter element to search: ";
    cin >> key;

    int low = 0, high = n - 1;

    while (low <= high) {
        int mid = (low + high) / 2;

        if (a[mid] == key) {
            cout << "Element found at position " << mid + 1 << endl;
            return 0;
        }
        else if (key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
    cout << "Element not found" << endl;
    return 0;
}
```

14. Write a C++ program to implement any two sorting algorithms and compare their number of comparisons for the same input array.

```cpp
int main14() {
    int a[100], b[100], c[100], n;
    int bubbleComparisons = 0;
    int selectionComparisons = 0;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        b[i] = a[i];
        c[i] = a[i];
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            bubbleComparisons++;
            if (b[j] > b[j + 1])
                swap(b[j], b[j + 1]);
        }
    }

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            selectionComparisons++;
            if (c[j] < c[minIndex])
                minIndex = j;
        }

        swap(c[i], c[minIndex]);
    }
    cout << "Bubble Sort comparisons = " << bubbleComparisons << endl;
    cout << "Selection Sort comparisons = " << selectionComparisons << endl;

    return 0;
}
```

15. Write a C++ program to implement any three sorting algorithms and display their Best, Average, and Worst-case time complexities as comments/output.

```cpp
int main15() {
    cout << "Bubble Sort: Best O(n), Average O(n^2), Worst O(n^2), Space O(1)\n";
    cout << "Insertion Sort: Best O(n), Average O(n^2), Worst O(n^2), Space O(1)\n";
    cout << "Selection Sort: Best O(n^2), Average O(n^2), Worst O(n^2), Space O(1)\n";

    return 0;
}
```